Changelog

v1.2.0 - Prefederation — 2026-11-11

- **Coverage after every route.** Under the route: which words of the task the chosen path covers, which the map knows but this route skipped (with the concept that fits them), and which are not covered.
- **Where the hierarchy breaks.** For the uncovered part (with its neighboring word, e.g. "android app") the app walks down the levels with the observer lens: no topic fits (best TF-IDF score below 0.2) → propose a new topic in the best domain; a topic fits but no concept → propose a new concept under it. No model and no network: it runs in the dashboard for everyone.
- **Fill the gap.** Builds a generation request for only the missing levels (with the task, the uncovered phrase and the target domain or topic). With your API key it generates, adds the pack and routes again in one step; inside Claude it sends the request to Claude; otherwise it copies the request. **Add pack and route again** adds the pasted pack and re-routes.
- **Generated packs now route.** Topics, concepts and skills from packs added in the app join the in-page routing index (flat and observer lens); words the map has never seen get the largest idf. Removing a pack takes them out. The public server's map is unchanged.
- **Copy buttons** on the route, details, cost line, embedding values, messages and every text field.
- Backup of v1.1.0: branch `release/v1.1.0-prefederation` .

v1.1.0 - Prefederation — 2026-11-11

- **Cost estimate before running.** Every routed skill, agent and MCP server shows its estimated cost on a small orbit beside the node; the line under the task gives the whole harness (input and output tokens, dollars).
- Skills are fixed: the tokens of their spec in the harness. Agents depend on role and loops: primary 4 work turns, checker 1 review turn, support only on request; each turn re-sends CLAUDE.md, the agent's prompt, the skill specs and the tool

definitions of the servers it holds, and history grows. MCP servers depend on the task: tool definitions (120 tokens per tool, an assumption) plus expected calls (the server's tools whose names share a word with the task, at least one) at 1,000 tokens per result.
- Model picker (Sonnet 5.5 default, Opus 5.5, Haiku 5.5, Fable 5.1) at Anthropic list prices checked 2026-11-10, no prompt caching (an upper estimate); a Loops box for maker → checker rounds; a Cost checkbox to hide the labels.
- Send to Claude Code adds `cost.json` (every part, with the assumptions) and an "Estimated cost" section in CLAUDE.md.

v1.0.0 - Prefederation — 2026-11-10 (first public release)

- One version for everything: the Maxey0 Context Plane server, the Maxey0 Context Plane Graph (the MCP App), the plugin and the paper are all v1.0.0 - Prefederation. Earlier entries keep their internal build numbers (server 6.x, app 1.x, plugin 0.x); v1.0.0 is internal build 6.7.1 / app 1.13 / plugin 0.5.0 plus the Context Plane paper in the research library.
- "Prefederation": no sign-in. Next release adds identity federation and a small hosted allowance for people without a Claude account.
- Phones: the Connect to Claude button moves to the bottom-right corner with a × to hide it to a small "Claude" tab on the right edge; tapping the tab brings it back (remembered in this browser). The button can be dragged anywhere on a phone screen; it snaps to the nearer side edge and keeps its place. The button always reads "Connect to Claude" (it shows the model only while your own API key is in use).
- The Context Plane paper is now in the repository (`research/papers/`) and served at `/research/context-plane` .

6.7.1 — 2026-11-10

- `/docs` : every doc (overview, user guide, MCP server and API reference, architecture, performance, development, changelog, privacy) as a web page, a PDF and a Markdown download. Built by `tools/site/build_site.py` , served as Workers static assets (no storage charge).

- `/research` : research library with a summary and a question-and-answer frame per paper, the PDF embedded in the page and downloadable. Starts with the Context Plane paper and the four Maxey0 papers (SCW, System One, Logical SCWs, execution-space control). The Context Plane PDF stays out of git and is copied in locally before deploy.
- Routes `maxey0.com/docs*` , `maxey0.com/research*` (take effect only if maxey0.com has a proxied DNS record).
- Home page: orange "Use Maxey0 inside Claude" button opens the app's Connect dialog (`/app#connect`); Docs and Research buttons.
- README redesigned.

6.7.0 (app 1.13, plugin 0.5.0) — 2026-11-10

- Landing page at `/` for browsers (unaltered Human0 logo at `/logo.png`); JSON server info via `?format=json` or non-HTML Accept.
- Custom domain `app.maxey0.com` (wrangler route; Cloudflare creates the DNS record and certificate on deploy).
- App: Maxey0 · by Human0 branding; "Connect to Claude" dialog (default: Maxey0 inside your Claude as a custom connector; or your own API key held in page memory only, never stored or sent to our server, with a model picker); Send to Claude Code (zip harness: CLAUDE.md, .mcp.json, .claude/agents, route.json).
- /app CSP allows connect-src https://api.anthropic.com only for BYOK calls.

Server 6.6.0 · App 1.12

- Embedding values of a selected node in Details, each labelled with what it is: the TF-IDF vector routing uses (non-zero terms and weights, length 1); the bge-m3 vector routing uses (1024 values, centred, unit; first 8 shown, Copy all) with its nearest nodes per level by dense cosine; the 3D field place (SVD axes 1–3: x, y, z); the 3D drawn place on its level donut seen from above; with an observer, the observer's value for the node (a and √a), its observed 3D field place, observed drawn place and the 3D displacement; with a lens route that chose it, flat score and rank → ψ and rank, drift, weight. Previously only x, y of the field and √a were shown, labelled "seen from above" in both views.

- Server: `GET /embedding?key=<node>` returns a node's bge-m3 vector and its dense nearest nodes per level (CORS, cached an hour).

Server 6.5.2 · App 1.11

- Performance (11): budget and checks in docs/PERFORMANCE.md. Page load ~25% faster (agent ring layout counts skills once per layout instead of once per concept; agent angles in one pass over the edges; gate lists sort only candidate servers) and the page's lens 4× faster (cached key counts, per-node constants computed once), with identical layouts, gates and routes. Budget tests in app (page size, load, frame, lens, heap) and server (CPU per route).

Server 6.5.1

- Dense defaults calibrated on the live server (bge-m3 index built): β = 0.7, match threshold 0.25. 300 skill labels: skill@1 0.933 (TF-IDF) → 0.987, topic@1 0.950 → 0.993. 20 realistic tasks: correct top skill 13/19 → 15/19, correct within top 3 14/19 → 17/19; 10 off-map requests all below the threshold (max 0.239). Observer lens with dense, same β (live, after the CPU fix): skill@1 0.923 → 0.983, topic@1 0.943 → 0.990, tasks top 3 16/19 → 17/19; it also puts the server-log anomaly spotter first. Details in docs/ARCHITECTURE.md.
- Lens speed: per-candidate constants computed once per selected node, node vectors normalised once per isolate, key counts cached. TF-IDF lens ~80 ms → ~17 ms CPU (node timing); results unchanged (fixture). The dense lens hit Cloudflare's resource limit (error 1102) on 6.5.0; now ~22 ms.
- Index warming keeps a progress bitmap, so a step never re-embeds chunks a lagging KV list does not show yet.
- Local test config moved to `wrangler.local.jsonc`; `npx wrangler deploy` no longer warns about multiple environments.

Server 6.5.0 · App 1.10

- Dense second signal (15): BAAI bge-m3 embeddings from Cloudflare Workers AI (binding `AI`), next to TF-IDF in one inner-product space: = (1 − β)·TF-IDF cosine

+ β·dense cosine. Node embeddings (all 3,986 nodes) are built once per graph digest in steps of 8 chunks of 50 and kept in KV (binding `DENSE`); embeddings are centred on the mean node embedding and normalised. The task is embedded per route. With no binding, or before the index exists, routing is TF-IDF only and the response says why.
- route_task and search take `dense` (default true), `dense_weight` (β, default 0.5 until calibrated) and `min_score`; responses carry `signal` (weights, model) and `dense_status`. Flat paths now include their `score`.
- The lens runs in the hybrid space; with dense, ψ is evaluated for the 300 best candidates of a level by flat hybrid score (all children of structural levels). Without dense the lens is bit-for-bit the TF-IDF lens (fixture test).
- New endpoints: `GET /dense` (status), `POST /dense` (one warm step, idempotent), `POST /route` (read-only routing for the app, CORS).
- App: routes in the page at once (TF-IDF), then asks the server; a route with the dense signal replaces the local one. **Dense signal** checkbox and a readout of which signal produced the route; saved with the state.
- Local tests run without Cloudflare credentials: `npx wrangler dev -c wrangler.local.jsonc --port 8794`.

Server 6.4.0 · App 1.9

- Fix (server + app): a word that is also an Object.prototype key ('constructor' is in the map's vocabulary) turned similarity scores into NaN for any task or node containing it (skill `symsys-automata-builder`). Vectors are now read by own keys only (`Object.hasOwn`). Test added on both sides.
- Drift in one shared 3D frame: the observer's view is now in the same field coordinates as the view from above. Each node keeps its 3D direction from the observer and its distance becomes (1 − √a) × the level's farthest distance, so displacement = observed − home is the node's own drift, in all three axes. Depth is framed per level (deepest node reaches the donut's top or bottom) instead of saturating at the rim.
- Color scale (10): with an observer and Embedding field on, nodes are colored by observer value √a on a perceptually even scale, with a legend in the graph's corner; drift arrows are green for + drift and red for − drift. **Observer color** toggles it; saved with the state.

- Accessibility (12): the graph takes keyboard focus. Left/Right move along a level, Down/Up to the node below/above, Page Up/Down change level, Home/End, Escape clears; a live region reads each node out. Active buttons darkened to meet WCAG AA text contrast (4.18 → ≥ 4.5); a contrast check over every visible text element is part of the tests.
- OWL 2 + SHACL rebuilt for 6.x (13): `ontology/` (11 domains, 128 topics, 3D field places on every node, drift vocabulary with SHACL rules) built by `tools/owl/build_owl.py`; parity, SHACL, negative controls and OWL 2 RL reasoning pass. See ontology/README.md.

Server 6.3.0 · App 1.8

- Observer lens continues below the skill: agents (children of the chosen skills) and MCP servers (gated to the chosen agents), chosen by the same ψ. λ agent = 0.9 (calibrated, held-out 0.463 primary-agent@1 vs 0.403 at 0.5); λ MCP = 0.9 (no ground truth).
- Drift: every chosen node records flat score, flat rank, lens rank, drift = ψ − flat and alignment with the observer; every level records task in/out and the share explained. route_task (lens) returns `agents`, `mcp_servers` and `drift`: SCW-drift per node, SCW-level per level, SCW-path around all levels.
- App: the lens readout shows drift and explained task per level; **Drift arrows** (field view) draw a ghost where each chosen node sits seen from above and an arrow to where the observer moved it, turning with the graph; **Export drift SCW** downloads the path record with home/observed 3D places and displacements.
- App = server checked value for value (ids, weights, ψ, drift, ranks, task norms) on 15 tasks through all five levels.
- Domain labels are now drawn (cpDrawDomains existed but was never called).

App 1.7

- Node size follows density: each level's node radius is 0.3 x the mean spacing on its donut (donut area / visible node count, times zoom), between 1.4 and 13 px. Fewer visible nodes (an isolated path, a filter) give bigger nodes.

- Embedding field: each level is framed on its own. Field coordinates have no meaning by themselves, so a level is centred on its own centroid and scaled by one factor (x, y and depth together) until its farthest node reaches the outer rim. Directions between a level's nodes and the order of their distances are kept; the hole adds a radial stretch near the centre.

Server 6.2.0 · App 1.6

- Embedding field is now 3D: the builder keeps the third principal axis (`field.z` , with `field.variance_share`). Axes 1–3 carry 8.1%, 7.3% and 6.2% of the variance of the 24-dim SVD space, so 3D shows 21.5% against 15.4% for 2D. `nearest` returns `z` with `xy` .
- Every level is a 3D donut: inner hole at 30% of the radius and a thickness of ±45. Axes 1–2 place a node around and across the ring, axis 3 places it in the thickness. Every node, dragged or not, is kept inside its donut.
- Drag moves a node in 3D parallel to the screen, so it follows the pointer from any angle (it used to stop or jump when a level was seen edge-on) and stays exactly where it is dropped. Saved layouts keep the depth.

App 1.5

- Level-edge toggles: Domain→Topic, Topic→Concept, Concept→Skill, and Related concepts (dashed, off by default), alongside the Skill→Agent and Agent→MCP gate toggles.
- Domains tab in the side list (11 domains).
- Topics generated in the app get a suggested domain (nearest topic by TF-IDF), shown as suggested.
- Saved state now keeps router mode, field, domain filter, edge toggles and browse filters.
- Related-concept pairs from the graph are drawn when enabled.

Server 6.1.0 · App 1.4

- **Domains are the sixth level of the graph:** one node per domain on a plane above the topics, linked to its topics, with a Domains count in Metrics and a **D** label button. The floating domain captions are gone.
- **Embedding values are visible:** Details shows each node's field location, its value from the current observer, and its nearest nodes on every level; nodes the observer barely explains fade in the embedding field.
- **Domain vectors** are now the centroid of their topics; a domain ranks in `nearest` as its nearest topic.
- `open_context_plane_graph` takes `domain`, `mode` and `field`, and the app applies them.

App 1.3

- **Domains in the graph:** each topic carries its domain's colour, domains are labelled where their topics are, and a Domains toggle.
- **Observer lens in the app:** a Flat / Observer lens router in Task, computing the same chain as the server's `route_task mode lens` (shared test fixture), with a readout of weights, signed support and ruled-out candidates.
- **Embedding field follows the observer:** above the graph it shows the shared field; after a lens route, or a domain, topic or concept choice, each plane below the observer is laid out from where it stands. The current observer is shown beside the toolbar.

Server 6.0.0 · App 1.2 · Plugin 0.4.0

- **Domains and topics rebuilt from the concepts up.** 11 new domains replace the 5 old ones; 128 topics were formed from what their concepts, skills, agents and MCP servers have in common (all 711 concepts, 1,998 skills, 1,021 agents and 117 servers unchanged). Domain endpoints are now `/mcp/<domain id>` for the 11 domains. Each concept records the topic it came from (`source_topic`).
- **Observer-lens routing.** `route_task` takes `mode: "lens"` : topics are chosen first, then concepts and skills are valued from the chosen nodes with signed support (several chosen nodes add as one superposed observer), and the answer

carries the full observer chain. `domains` starts the observer at one or more domains. Flat ranking stays the default.
- App: a **Domain** filter in Browse & select.
- New topic packs get their domain from `regroup.pack_domains` in the spec.

Server 5.2.0 · App 1.1

- **Shared embedding field.** Every node on all six levels (domain, topic, concept, skill, agent, MCP server) now has a vector in one TF-IDF space and a location in one 2-D field (truncated SVD, same scale for every level). `graph.json` carries the locations; `?index=1` carries all vectors.
- New tool `nearest`: the nearest nodes of any level to a node or a piece of text in that field.
- App: **Embedding field** places every node at its field location on its level.

Server 5.1.2

- `get_harness` description states what it returns, without directing the model to other tools.

Server 5.1.1

- Every tool declares `annotations.title` for directory listings.
- App: the task panel shows the route only (agent, server and model choice happen in the Maxey0 orchestrator); legend explains rotate, roll, pan and node drag.

Server 5.1.0 · App 1.0 · Plugin 0.3.1

- **MCP App.** `open_context_plane_graph` shows the graph inside the conversation in hosts that support MCP Apps; the app takes the task Claude passes in, can go full screen, and sends topic-generation requests to the conversation.
- **Web app** served by the server at `/app`.

- **Plugin** renamed `maxey0-context-plane-graph`.

Server 5.0.0

- **Maxey0 is one topic** with eight concepts (was three topics). Former topic ids `maxey0-runtime`, `maxey0-memory-context` and `maxey0-assurance` are replaced by `maxey0`.

Server 4.0.0

- **Two new domains:** Foundations (Location and Place, Environments and Deployment, Time, State, Systems, Infrastructure) and Finance (16 topics, from Accounting to Crypto).
- **Five new work topics:** UI and Front-End Development, Security and Identity, Real Estate and Property, Simulation and Digital Twins, Travel and Outdoors.
- **30 new MCP servers.**
- **Topic generation:** the `topic-graph-generator` prompt and the `validate_topic_pack` tool.

App 1.0

- One app: task routing with the server's method, best route highlighted, result panel that hides to one line.
- Level rings, spread paths, node dragging that carries the path below it.
- Rotation in any direction, roll, pan, 720° tumble, Fit view.
- Generated topics: add, validate, keep or remove each, view new topics alone.
- State records the map version and the view.

Server 3.x

- One endpoint per domain plus one for all; `/graph.json` for browser apps; checker agents inherit their primary's servers; permanent MaxeyN keys.